

Glosario esencial de administración y PL/SQL

Unidad 2 · Administración de DBMS

Oracle Database · procedimientos y objetos PL/SQL

Propósito de esta guía

Comprender, escribir y revisar los elementos trabajados en la Unidad 2. Cada término conecta su propósito con la sintaxis, un ejemplo de los guiones SQL y las precauciones que evitan errores frecuentes en Oracle.

Cómo utilizar esta guía

Esta guía reúne los conceptos, instrucciones y objetos trabajados en las seis presentaciones de la Unidad 2 y en sus cinco guiones SQL. Puede utilizarse de dos maneras:

- **Lectura progresiva:** seguir los bloques en orden, desde la seguridad y los procedimientos hasta los triggers.
- **Consulta rápida:** buscar un término en el índice o en el glosario final y revisar el ejemplo asociado.

Regla de lectura

Antes de ejecutar un script identifique si modifica datos, crea un objeto o controla una transacción. Pruebe en un esquema de laboratorio y compruebe el estado de la tabla después de cada COMMIT o ROLLBACK.

```
DECLARE
  v_nombre VARCHAR2(50);
BEGIN
  SELECT NOMBRE_USUARIO INTO v_nombre
  FROM CACB_USUARIO WHERE COD_USUARIO = 1;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    v_nombre := 'SIN REGISTRO';
END;
```

1.3 Procedimiento almacenado

Un procedimiento compilado y almacenado recibe parámetros y realiza acciones de mantenimiento. CREATE OR REPLACE permite actualizar su definición.

```
CREATE OR REPLACE PROCEDURE CACB_CARGA_USUARIOS(
  COD_USUARIO_P NUMBER, NOMBRE_USUARIO_P VARCHAR2)
IS
BEGIN
  INSERT INTO CACB_USUARIO(COD_USUARIO, NOMBRE_USUARIO)
  VALUES(COD_USUARIO_P, NOMBRE_USUARIO_P);
END;
```

Diferencia esencial

Un procedimiento realiza una acción; una función retorna un valor. Ambos pueden tener variables locales, pero solo la función declara RETURN.

1 Seguridad de la información y PL/SQL

1.1 Seguridad de la información

La seguridad se estudia mediante tres propiedades: confidencialidad, disponibilidad e integridad. En una base de datos se relacionan con usuarios y controles de acceso, respaldo y recuperación, y diseño con restricciones.

Idea clave. La seguridad de la información protege la información dondequiera que esté; la seguridad informática se limita al medio informático.

1.2 PL/SQL y bloque estructurado

PL/SQL es el lenguaje procedural de Oracle. Un bloque puede incluir DECLARE para variables, BEGIN para instrucciones, EXCEPTION para errores y END para cerrar.

1.4 Parámetros, tipos y validaciones

Los parámetros IN reciben información, OUT entregan un valor al programa que llama y IN OUT permite ambas operaciones. Si no se indica el modo, el parámetro es IN. Para evitar que una definición quede desactualizada cuando cambie una columna, el tipo puede tomar como referencia la tabla mediante `tabla.columna%TYPE`.

Procedimiento con validación

```

CREATE OR REPLACE PROCEDURE CACB_MODIFICA_USUARIO(
  p_codigo CACB_USUARIO.COD_USUARIO%TYPE,
  p_nombre CACB_USUARIO.NOMBRE_USUARIO%TYPE)
IS
BEGIN
  IF p_nombre IS NULL OR TRIM(p_nombre) IS NULL THEN
    RAISE_APPLICATION_ERROR(-20001, 'EL NOMBRE ES OBLIGATORIO');
  END IF;
  UPDATE CACB_USUARIO
    SET NOMBRE_USUARIO = UPPER(TRIM(p_nombre))
  WHERE COD_USUARIO = p_codigo;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(-20002, 'USUARIO NO ENCONTRADO');
  END IF;
END;

```

La validación debe ocurrir antes del DML cuando sea posible. Después de una operación se puede revisar `SQL%ROWCOUNT` para distinguir una modificación real de una llamada cuyo código no existe. El procedimiento no decide por sí solo si la aplicación confirma la transacción; esa decisión debe quedar documentada y ser coherente con el bloque que lo invoca.

2 Concurrency, bloqueos y transacciones

2.1 Concurrency

La concurrencia ocurre cuando varios usuarios trabajan sobre recursos compartidos. `LOCK TABLE` establece un modo de bloqueo para coordinar operaciones.

Sintaxis: `LOCK TABLE nombre_tabla IN ROW EXCLUSIVE MODE`

`ROW SHARE` permite acceso simultáneo y evita un bloqueo exclusivo de la tabla. `ROW EXCLUSIVE` también impide el modo `SHARE` y se obtiene automáticamente con `INSERT`, `UPDATE` o `DELETE`. `SHARE` permite consultas, pero no actualizaciones. `EXCLUSIVE` permite consultas y bloquea las demás actividades.

Control de concurrencia

```

LOCK TABLE CACB_USUARIO IN ROW EXCLUSIVE MODE;
INSERT INTO CACB_USUARIO VALUES (1, 'CARLOS');
COMMIT;

```

2.2 COMMIT, ROLLBACK y SAVEPOINT

`COMMIT` confirma los cambios de la transacción. `ROLLBACK` deshace el trabajo actual y `SAVEPOINT` marca un punto al que se puede retroceder. Una consulta no coloca un bloqueo de tabla y una tabla bloqueada sigue pudiendo

consultarse. Una transacción comienza con el primer DML y termina con `COMMIT`, `ROLLBACK` o un DDL que produzca confirmación implícita. Para conservar varias operaciones como una sola unidad, es preferible que el bloque llamador controle la confirmación.

Punto de recuperación

```

BEGIN
  SAVEPOINT antes_de_actualizar;
  UPDATE CACB_USUARIO SET NOMBRE_USUARIO = 'ANA'
  WHERE COD_USUARIO = 2;
  IF SQL%ROWCOUNT = 0 THEN
    ROLLBACK TO antes_de_actualizar;
  END IF;
  COMMIT;
END;

```

Transacción y compilación

No confundas confirmar la transacción con compilar el procedimiento. Son acciones diferentes y ocurren en momentos distintos.

2.3 Excepciones

Una excepción es un error PL/SQL que puede producirse implícita o explícitamente. RAISE transfiere el control al manejador. RAISE_APPLICATION_ERROR genera un error propio con códigos entre -20 000 y -20 999.

EXCEPTION

```
WHEN PROGRAM_ERROR THEN
    RAISE_APPLICATION_ERROR(-6501, 'ERROR INTERNO');
WHEN OTHERS THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20010, 'ERROR DE CARGA');
END;
```

2.4 Catálogo de errores

Oracle informa cada error con un código numérico y un mensaje. En PL/SQL se pueden capturar excepciones con nombre, consultar el código mediante SQLCODE y el texto mediante SQLERRM, o convertir una condición propia en

una excepción con RAISE_APPLICATION_ERROR. No existe una lista finita de nombres para todos los errores de Oracle: el catálogo completo depende del motor y de sus componentes. La siguiente tabla reúne los códigos que aparecen con mayor frecuencia al trabajar con procedimientos, funciones, transacciones y triggers.

Código o nombre	Categoría	Cuándo ocurre
NO_DATA_FOUND / ORA-01403	Predefinida	SELECT INTO no encuentra ninguna fila.
TOO_MANY_ROWS / ORA-01422	Predefinida	SELECT INTO devuelve más de una fila.
DUP_VAL_ON_INDEX / ORA-00001	Predefinida	Se repite una clave primaria o una restricción única.
VALUE_ERROR / ORA-06502	Predefinida	Conversión inválida, valor nulo no permitido o dato demasiado largo.
ZERO_DIVIDE / ORA-01476	Predefinida	Se intenta dividir por cero.
INVALID_NUMBER / ORA-01722	Predefinida	Un texto no puede convertirse al tipo numérico esperado.
CASE_NOT_FOUND / ORA-06592	Predefinida	Un CASE sin ELSE no encuentra coincidencia.
CURSOR_ALREADY_OPEN / ORA-06511	Predefinida	Se abre un cursor que ya estaba abierto.
INVALID_CURSOR / ORA-01001	Predefinida	Se usa un cursor cerrado, inexistente o no válido.
PROGRAM_ERROR / ORA-06501	Predefinida	Error interno de ejecución de PL/SQL.
STORAGE_ERROR / ORA-06500	Predefinida	PL/SQL no puede reservar memoria suficiente.
LOGIN_DENIED / ORA-01017	Predefinida	Credenciales inválidas al iniciar sesión.
NO_PRIVILEGES / ORA-01031	Predefinida	El usuario no tiene privilegios para la operación.
TABLE_OR_VIEW_NOT_FOUND / ORA-00942	SQL nombrada	La tabla o vista no existe o no es visible para el usuario.
TIMEOUT_ON_RESOURCE / ORA-00051	SQL nombrada	Se agota el tiempo esperando un recurso bloqueado.
MUTATING_TABLE / ORA-04091	Trigger	Un trigger por fila consulta o modifica su propia tabla en mutación.
CUSTOM / ORA-20xxx	Personalizada	Regla de negocio definida por el equipo mediante RAISE_APPLICATION_ERROR.

Los nombres predefinidos se pueden capturar directamente en el bloque `EXCEPTION`. Los errores que no tienen nombre predefinido se pueden asociar con `PRAGMA`

`EXCEPTION_INIT` o manejar en `WHEN OTHERS`. El código de usuario debe estar entre -20 000 y -20 999; el mensaje puede tener hasta 2.048 caracteres.

Estructura básica de una excepción personalizada

```
CREATE OR REPLACE PROCEDURE CACB_CARGA_USUARIO(
  COD_USUARIO_P CACB_USUARIO.COD_USUARIO %TYPE,
  NOMBRE_USUARIO_P CACB_USUARIO.NOMBRE_USUARIO %TYPE)
IS
  USUARIO_DUPLICADO EXCEPTION;
  PRAGMA EXCEPTION_INIT(USUARIO_DUPLICADO, -20001);
BEGIN
  IF COD_USUARIO_P IS NULL THEN
    RAISE_APPLICATION_ERROR(-20002, 'EL CÓDIGO ES OBLIGATORIO');
  END IF;

  INSERT INTO CACB_USUARIO(COD_USUARIO, NOMBRE_USUARIO)
  VALUES(COD_USUARIO_P, UPPER(TRIM(NOMBRE_USUARIO_P)));

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    RAISE USUARIO_DUPLICADO;
  WHEN VALUE_ERROR THEN
    RAISE_APPLICATION_ERROR(-20003, 'FORMATO DE DATO NO VÁLIDO');
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(
      -20010,
      'ERROR DE CARGA ' || SQLCODE || ': ' || SQLERRM);
END;
```

En la plantilla, `DUP_VAL_ON_INDEX` identifica una clave repetida, `VALUE_ERROR` identifica un dato incompatible y `WHEN OTHERS` funciona como último control. Si se relanza

un error, conserve su causa y registre `SQLCODE` y `SQLERRM`; no oculte el diagnóstico con un mensaje genérico.

Asociar un código Oracle sin nombre

```
DECLARE
  RECURSO_BLOQUEADO EXCEPTION;
  PRAGMA EXCEPTION_INIT(RECURSO_BLOQUEADO, -51);
BEGIN
  LOCK TABLE CACB_USUARIO IN EXCLUSIVE MODE;
EXCEPTION
  WHEN RECURSO_BLOQUEADO THEN
    RAISE_APPLICATION_ERROR(-20020,
      'NO SE PUDO OBTENER EL BLOQUEO: ' || SQLERRM);
END;
```

Orden recomendado del manejador

Coloque primero las excepciones específicas y deje `WHEN OTHERS` al final. Registre `SQLCODE` y `SQLERRM` y relance el error con `RAISE` o `RAISE_APPLICATION_ERROR`.

3 Funciones PL/SQL

Una función es un bloque nombrado que devuelve un valor y puede llamarse como parte de una expresión. `SELECT INTO` lleva el resultado de una consulta a una variable. En los ejemplos se captura `NO_DATA_FOUND` cuando no existe el empleado.

Función de sueldo

```
CREATE OR REPLACE FUNCTION CACB_OBTENER_SUELDO(EMPLOYEE_ID_F NUMBER)
RETURN NUMBER IS
    SUELDO_OBTENIDO NUMBER := 0;
BEGIN
    SELECT SALARY INTO SUELDO_OBTENIDO
    FROM OEHR_EMPLOYEES WHERE EMPLOYEE_ID = EMPLOYEE_ID_F;
    IF SUELDO_OBTENIDO IS NULL THEN SUELDO_OBTENIDO := 0; END IF;
    RETURN SUELDO_OBTENIDO;
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN -1;
END;
```

Restricción

Una función llamada desde SQL no debe ejecutar INSERT, UPDATE o DELETE, ni llamar a un subprograma que rompa esa restricción.

3.1 Funciones de transformación y valores nulos

Las funciones también pueden concentrar reglas repetitivas, como normalizar nombres o construir un correo institucional. SUBSTR extrae posiciones, UPPER estandariza mayúsculas y NVL reemplaza un valor nulo. La regla debe definir qué ocurre cuando el nombre o el apellido tiene menos letras.

Construcción de correo

```
CREATE OR REPLACE FUNCTION CACB_GENERA_CORREO(
    p_nombre VARCHAR2, p_apellido1 VARCHAR2, p_apellido2 VARCHAR2)
RETURN VARCHAR2 IS
BEGIN
    RETURN LOWER(SUBSTR(TRIM(p_nombre),1,2) ||
        SUBSTR(TRIM(p_apellido1),1,2) || '.' ||
        REPLACE(TRIM(p_apellido2), ' ', '')) || '@testsql.cl');
END;
```

Una función que se use en una consulta debe ser determinista respecto de sus entradas: para resolver duplicados, conviene que el procedimiento de carga consulte los correos existentes y agregue un sufijo antes de insertar.

4 Secuencias

Una secuencia genera enteros únicos para uno o varios usuarios y puede alimentar una clave primaria. Sus valores son independientes de las tablas y de la confirmación de la transacción; por eso pueden existir saltos.

Creación de la secuencia de usuarios

```
CREATE SEQUENCE CACB_PK_USUARIO
START WITH 1 INCREMENT BY 1
MINVALUE 1 MAXVALUE 2500
CACHE 200 NOCYCLE;
```

START WITH fija el primer valor; INCREMENT BY define el paso; MINVALUE y MAXVALUE fijan límites. CYCLE reinicia al alcanzar MAXVALUE y NOCYCLE

evita reutilizar valores. NEXTVAL incrementa y devuelve el siguiente valor; CURRVAL lee el valor actual de la sesión después de usar NEXTVAL.

Uso en un INSERT

```
INSERT INTO CACB_USUARIO(COD_USUARIO, NOMBRE_USUARIO)
VALUES(CACB_PK_USUARIO.NEXTVAL, 'CARLOS');
```

El valor de una secuencia no se recupera con ROLLBACK: una inserción cancelada puede dejar un número libre. CACHE mejora el rendimiento, pero puede producir saltos si la instancia se reinicia. Por eso una secuencia identifica filas; no sirve para contar cuántos registros existen ni garantiza que no haya intervalos.

5 Cursores PL/SQL

Los cursores manejan el conjunto de registros devuelto por SELECT. El cursor implícito es gestionado por Oracle para una fila; el explícito se declara para varias filas o para reutilizar la consulta. La declaración no lleva INTO.

Cursor explícito

```
DECLARE
  CURSOR c_usuarios IS
    SELECT COD_USUARIO, NOMBRE_USUARIO
    FROM CACB_USUARIO ORDER BY COD_USUARIO;
BEGIN
  FOR r IN c_usuarios LOOP
    DBMS_OUTPUT.PUT_LINE(r.NOMBRE_USUARIO);
  END LOOP;
END;
```

Orden de las filas

Si el procesamiento depende del orden, incluya ORDER BY en la consulta del cursor. No coloque INTO en la declaración del cursor.

5.1 Atributos y cursores parametrizados

Después de abrir o recorrer un cursor se pueden consultar %FOUND, %NOTFOUND, %ROWCOUNT e %ISOPEN. Un cursor parametrizado permite reutilizar la misma consulta para distintos códigos sin concatenar SQL.

Cursor con parámetro

```
DECLARE
  CURSOR c_usuarios(p_minimo NUMBER) IS
    SELECT COD_USUARIO, NOMBRE_USUARIO
    FROM CACB_USUARIO
    WHERE COD_USUARIO >= p_minimo
    ORDER BY COD_USUARIO;
BEGIN
  FOR r IN c_usuarios(2) LOOP
    DBMS_OUTPUT.PUT_LINE(r.COD_USUARIO || ': ' || r.NOMBRE_USUARIO);
  END LOOP;
END;
```

El ciclo FOR r IN cursor abre, obtiene y cierra automáticamente el cursor. El cursor explícito con OPEN, FETCH y CLOSE es útil cuando se necesita controlar cada etapa, pero exige cerrar el recurso aun cuando ocurra una excepción.

6 Triggers e integridad

Un trigger es un bloque PL/SQL asociado a una tabla que se dispara por INSERT, UPDATE o DELETE. Sus componentes son nombre, evento y cuerpo; WHEN es opcional. FOR EACH ROW hace que se ejecute una vez por cada fila.

Cálculo automático del promedio

```
CREATE OR REPLACE TRIGGER CACB_PROMEDIO_AUTO
BEFORE INSERT ON CACB_DETALLE_NOTAS
FOR EACH ROW
BEGIN
  :NEW.PROMEDIO := ROUND((:NEW.NOTA1+:NEW.NOTA2+:NEW.NOTA3)/3,1);
END;
```

:NEW representa los valores nuevos y :OLD los anteriores. Los triggers sirven para mantener restricciones de integridad complejas, auditar información y avisar a otros programas. Un trigger no puede emitir COMMIT, ROLLBACK ni SAVEPOINT. Un trigger BEFORE puede corregir o completar :NEW; un trigger AFTER es apropiado para

registrar que la operación ya ocurrió. En un INSERT solo existe :NEW; en un DELETE solo :OLD; en un UPDATE están disponibles ambos. Las expresiones con nulos deben tratarse de forma explícita, porque una suma con una nota nula produce un promedio nulo.

Promedio tolerante a valores nulos

```
CREATE OR REPLACE TRIGGER CACB_PROMEDIO_AUTO
BEFORE INSERT OR UPDATE OF NOTA1, NOTA2, NOTA3
ON CACB_DETALLE_NOTAS
FOR EACH ROW
BEGIN
    :NEW.PROMEDIO := ROUND((NVL(:NEW.NOTA1,0) +
                           NVL(:NEW.NOTA2,0) +
                           NVL(:NEW.NOTA3,0)) / 3, 1);
END;
```

Antes de publicar un trigger, compruebe si debe actuar por fila o por sentencia, qué eventos lo activan y si puede generar recursión al modificar la misma tabla.

Transacción fuera del trigger

El trigger calcula o valida la fila; el procedimiento o bloque que lo invoca debe controlar la transacción.

7 Relaciones entre elementos

Necesidad	Elemento que corresponde
Encapsular una operación DML reutilizable	Procedimiento almacenado
Devolver un valor para una expresión SQL	Función PL/SQL
Generar claves únicas entre sesiones	Secuencia
Procesar varias filas de un SELECT	Cursor explícito
Completar o validar una fila automáticamente	Trigger
Coordinar acceso a una tabla compartida	LOCK TABLE
Confirmar o deshacer una unidad de trabajo	COMMIT / ROLLBACK

8 Ruta de revisión de un script

Para revisar un guion de la unidad, siga este orden:

1. Identifique los objetos que crea o reemplaza y verifique que el esquema tenga los privilegios necesarios.
2. Compruebe tipos, nombres y restricciones de las tablas antes de compilar procedimientos, funciones o triggers.
3. Separe la prueba de compilación de la prueba funcional: consulte los errores con **SHOW ERRORS** y luego ejecute casos válidos y casos límite.
4. Observe la transacción: anote dónde se bloquea, dónde se confirma y qué excepción provoca un rollback.
5. Valide el resultado con una consulta independiente y deje el esquema de laboratorio en un estado conocido.

Pruebas mínimas

Incluya al menos un registro válido, un código inexistente, un valor nulo, un duplicado de clave y una ejecución concurrente cuando el script utilice secuencias o **LOCK TABLE**. Estos casos muestran si la regla está implementada o solo funciona para el ejemplo feliz.

Glosario rápido

COMMIT Confirma la transacción y materializa sus cambios.

CURRVAL Devuelve el valor actual de una secuencia en la sesión.

Cursor explícito Cursor declarado para recorrer varias filas.

EXCEPTION Sección de un bloque donde se manejan errores.

FOR EACH ROW Ejecuta un trigger una vez por fila afectada.

Función PL/SQL Bloque nombrado que devuelve un valor.

LOCK TABLE Bloquea una tabla en un modo especificado.

NEXTVAL Incrementa una secuencia y devuelve el nuevo valor.

NO_DATA_FOUND Excepción producida cuando **SELECT INTO** no encuentra filas.

PL/SQL Lenguaje procedural incorporado en Oracle.

Procedimiento almacenado Subprograma compilado que realiza una acción.

RAISE Transfiere el control al manejador de excepciones.

RAISE_APPLICATION_ERROR Genera un error definido por el usuario.

ROLLBACK Deshace el trabajo de la transacción actual.

SAVEPOINT Marca un punto intermedio de recuperación.

Secuencia Objeto que genera números únicos.

Trigger Bloque asociado a una tabla que responde a un evento DML.

:NEW / :OLD Valores nuevo y anterior disponibles en un trigger.

ROW EXCLUSIVE Modo de bloqueo para operaciones DML por filas.

SELECT INTO Asigna el resultado de una consulta a una variable.

Fuentes internas consultadas

SQL: U2/codigos/U2_C1.SQL, U2/codigos/U2_C1-2.SQL, U2/codigos/U2_C2.SQL, U2/codigos/U2_C2-2.SQL y U2/codigos/U2_C4.SQL. **Presentaciones:** seis archivos disponibles en U2/presentaciones/.

Antes de ejecutar un script. Compruebe el objeto que modifica, revise la transacción que lo contiene, verifique excepciones y permisos, y pruebe primero con datos de laboratorio.